



امنیت داده و شبکه

نیم‌سال دوم ۱۴۰۴-۰۵

دکتر مهدی خرازی و دکتر مرتضی امینی

پاسخ‌دهنده: معین آعلی - ۴۰۱۱۰۵۵۶۱

تمرین چهارم

فهرست مسائل

۱	 مسئله ۱
۱	 پروتکل مبتنی بر رمزنگاری نامتقارن
۱	 پروتکل ایجاد کلید جلسه
۳	 پروتکل پرداخت PayDroid
۳	 توزیع کلید با سرور مرکزی

پاسخ مسئله‌ی ۱.

پروتکل مبتنی بر رمزنگاری نامتقارن

آ

- این پروتکل برای احراز اصالت کلاینت طراحی شده تا کاربر A ثابت کند همان کسی است که ادعا می‌کند.
- پیام اول $(A \rightarrow B : [ID]_A)$: کاربر A شناسه خودش را برای B می‌فرستد تا اعلام حضور کند و بگوید می‌خواهد ارتباط را شروع کند.
 - پیام دوم $(B \rightarrow A : N_B)$: کاربر B یک چالش (Challenge) به صورت یک نانس تصادفی (N_B) می‌فرستد. این کار برای جلوگیری از حمله Replay است تا مطمئن شود پاسخ کلاینت زنده و در لحظه است.
 - پیام سوم $(A \rightarrow B : E(PrvKey_A, N_B))$: کاربر A نانس دریافتی را با کلید خصوصی خودش رمز (در واقع امضا) می‌کند و پس می‌فرستد.

چون فقط A به کلید خصوصی‌اش $(PrvKey_A)$ دسترسی دارد و نانس هم جدید بوده، B با رمزگشایی این پیام توسط کلید عمومی A و تطبیق آن با N_B مطمئن می‌شود که در همین لحظه دارد با خود واقعی A صحبت می‌کند.

ب

این پروتکل در برابر حمله جعل هویت از طریق سرور واسط آسیب‌پذیر است. مشکل اینجاست که امضای A روی نانس، هیچ وابستگی به هویت مقصد (B) ندارد. اگر A بخواهد همزمان با یک سرور مخرب یا کاربر دیگر به نام C ارتباط برقرار کند، C می‌تواند نانسی که B برایش فرستاده را عیناً به عنوان چالش برای A بفرستد. A آن را امضا می‌کند و به C می‌دهد. حالا C امضای A روی نانس B را دارد؛ آن را به B می‌فرستد و خودش را به جای A جا می‌زند، بدون اینکه کلید خصوصی A را داشته باشد.

پروتکل ایجاد کلید جلسه

آ

- A فکر می‌کند کلید K'_{AB} را با B به اشتراک گذاشته، چون این کلید داخل یک پیام رمز شده با کلید بلندمدت K_{AB} آمده است؛ کلیدی که از نظر A فقط خودش و B آن را بلد هستند.
- B فکر می‌کند کلید K'_{AB} را با A به اشتراک گذاشته، چون در پیام سوم، A توانسته نانس را که B تعیین کرده بود (N_A که در پیام دوم داخل متن رمز بود) استخراج کند و حالا آن نانس را با خود کلید جلسه جدید (K'_{AB}) رمز کرده و پس فرستاده است. این یعنی A پیام دوم را باز کرده و کلید جدید را قبول دارد.
- A فکر می‌کند کلید K'_{AB} تازه است، چون خودش در پیام اول نانس N_A را فرستاده بود و در پیام دوم، همان نانس را به صورت رمز شده پس گرفته است. این یعنی پیام دوم قدیمی نیست و در پاسخ به درخواست فعلی او تولید شده است.
- B فکر می‌کند کلید K'_{AB} تازه است، چون خودش نانس N_A را از پیام اول گرفته، کلید جدید را کنارش گذاشته و فرستاده است. نانس N_A برای B نقش شناسه نشست فعلی را دارد. (البته این فرض ضعف اصلی پروتکل است، چون نانس را A تولید کرده نه B).

ب

چون پروتکل متقارن است و امکان اجرای معکوس (شروع از سمت B) وجود دارد، مهاجم C بدون داشتن کلید بلندمدت K_{AB} می‌تواند از خود A به عنوان یک Oracle استفاده کند تا پیام رمز شده‌ی مورد نظر را برایش تولید کند.

کاربر A قصد دارد یک نشست عادی با B شروع کند و نانس N_A را می‌فرستد. مهاجم این پیام را Intercept می‌کند:

$$A \rightarrow C(B) : A, N_A$$

مهاجم پیام بالا را نگه می‌دارد و یک اجرای معکوس (پروتکل رفرش از سمت B) با خود A باز می‌کند و همان نانس N_A را به عنوان چالش به A می‌فرستد تا خودش را جای B جا بزند:

$$C(B) \rightarrow A : B, N_A$$

حالا A در نقش پاسخ‌دهنده به اجرای معکوس، یک کلید تازه مانند K^* تولید کرده و آن را همراه با نانس N_A با کلید K_{AB} رمز می‌کند و می‌فرستد:

$$A \rightarrow C(B) : E(K_{AB}, [N_A, K^*])$$

مهاجم که به این رمز دست پیدا کرده، همین پیام را عیناً به عنوان پیام دوم اجرای اول به A برمی‌گرداند:

$$C(B) \rightarrow A : E(K_{AB}, [N_A, K^*])$$

از نظر A این پیام معتبر است، چون با کلید K_{AB} رمز شده و حاوی همان نانس اولیه خودش (N_A) است. پس فکر می‌کند پیام از طرف B آمده و تاییدیه نهایی را با کلید جدید K^* می‌فرستد:

$$A \rightarrow C(B) : E(K^*, N_A)$$

مهاجم این تاییدیه را هم عیناً به اجرای معکوس برمی‌گرداند تا آن نشست را هم تکمیل کند:

$$C(B) \rightarrow A : E(K^*, N_A)$$

در پایان این فرآیند، B اصلا در جریان ارتباط نبوده و روحش هم از این ماجرا خبر ندارد، اما A کاملا فریب خورده و فکر می‌کند کلید تازه K^* را با B به اشتراک گذاشته است، در حالی که این کلید فقط بین A و مهاجم (که نقش آینه را بازی کرده) قرار دارد. دلیل اصلی موفقیت این حمله، مشخص نبودن نقش فرستنده و گیرنده (هویت‌ها) در داخل بدنه پیام رمز شده است.

ج

بهترین کار این است که B هم یک نانس اختصاصی (N_B) تولید کند و در رمزگذاری پیام دوم مشارکت دهد، یا اینکه هویت‌ها صراحتا داخل پیام رمز شده بیایند:

$$B \rightarrow A : E(K_{AB}, [N_A, N_B, K'_{AB}])$$

$$A \rightarrow B : E(K'_{AB}, N_B)$$

اینطوری B هم مطمئن می‌شود پیام سوم در پاسخ به نانس جدید خودش تولید شده است.

پروتکل پرداخت PayDroid

آ

خیر، این پروتکل اصلا امن نیست. مشکل بزرگ این است که پیام حاوی هویت گیرنده (B) و مبلغ (X) به صورت آشکار فرستاده می‌شود و امضای دیجیتال A فقط روی X و n قرار دارد: $E(PrivKey_A, [X, n])$. مهاجم (مثلا خود Bob یا یک شنودکننده مثل C) پیام آلیس را در شبکه Intercept می‌کند. مهاجم هویت B را در بخش آشکار پیام حذف کرده و شناسه خودش (مثلا C) را قرار می‌دهد:

$$A \rightarrow S : A, C, X, n, Cert_A, E(PrivKey_A, [X, n])$$

وقتی پیام به سرور S می‌رسد، سرور امضا را با کلید عمومی آلیس چک می‌کند. امضا درست است (چون مبلغ X و نانس n تغییر نکرده‌اند). سرور هم بدون اینکه خبر داشته باشد، پول را به حساب C واریز می‌کند!

ب

برای حل این مشکل، باید هویت گیرنده (B) هم حتما داخل پکیج امضا قرار بگیرد تا کسی نتواند مقصد پول را در مسیر عوض کند:

$$A \rightarrow S : A, B, X, n, Cert_A, E(PrivKey_A, [B, X, n])$$

توزیع کلید با سرور مرکزی

آ

- خط ۱ ($A \rightarrow S : A, B$): کاربر A به سرور اعلام می‌کند که من (A) می‌خواهم با B یک ارتباط امن داشته باشم، لطفا یک کلید مشترک برای ما بساز.
- خط ۲ ($S \rightarrow A : E(K_A, R), E(K_B, R)$): سرور کلید تصادفی R را تولید می‌کند. یک نسخه از آن را با کلید مخفی A رمز می‌کند تا فقط A بفهمد، و نسخه دیگر را با کلید مخفی B رمز می‌کند و هر دو را به A می‌دهد.
- خط ۳ ($A \rightarrow B : E(K_B, R), E(R, M)$): کاربر A بخش مربوط به B را برایش می‌فرستد تا B هم به کلید R برسد. همزمان پیغام M را با کلید جلسه R رمز کرده و برای B ارسال می‌کند.

ب

هکر قادر است به پیغام M دسترسی پیدا کند. از آنجایی که این پروتکل هیچ مکانیزمی برای احراز اصالت سرور (مانند امضای دیجیتال سرور) و هیچ نانسی برای تضمین تازگی ندارد، در برابر جعل هویت سرور و Replay Attack آسیب پذیر است.

اگر هکر (C) از قبل یک کلید نشست قدیمی (R_{old}) و پیام رمز شده $(E(R_{old}, M_{old}))$ را به همراه تیکت B (یعنی $E(K_B, R_{old})$) شنود و ذخیره کرده باشد، می تواند مراحل زیر را اجرا کند:

کاربر A طبق روال درخواست خود را به سمت سرور می فرستد. هکر این پیام را Intercept می کند و اجازه نمی دهد به سرور واقعی برسد:

$$A \rightarrow C(S) : A, B$$

هکر خودش را جای سرور S جا می زند و پیام های قدیمی و لو رفته ی نشست قبلی را برای A بازتاب می دهد. از آنجا که پیام با کلید K_A رمز نشده، هکر می تواند یک کلید تصادفی دلخواه خودش $(R_{attacker})$ را با کلید K_A که احتمالا از قبل با یک حمله دیگر یا نشست قدیمی به دست آورده رمز کند، یا اگر فرض کنیم هکر کلیدهای قدیمی را دارد، پیام زیر را می فرستد:

$$C(S) \rightarrow A : E(K_A, R_{attacker}), E(K_B, R_{old})$$

کاربر A کلید $R_{attacker}$ را استخراج کرده، پیام جدید M را با آن رمز می کند و به همراه تیکت قدیمی B ارسال می کند:

$$A \rightarrow C(B) : E(K_B, R_{old}), E(R_{attacker}, M)$$

هکر پیام $E(R_{attacker}, M)$ را Intercept می کند. چون خودش کلید $R_{attacker}$ را تعیین کرده بود، به راحتی آن را رمزگشایی کرده و به متن آشکار پیغام M دسترسی پیدا می کند. این نشان می دهد که پروتکل بدون استفاده از نانس و احراز اصالت دقیق سرور کاملا ناامن است.